

Design Studio

AutoRoute Root-Cause Analysis

Why the router output is garbage vs Altium / KiCad / Flux.ai

Date: 2026-06-23 Project: single_axis_FOC_Field_Oriented_Control_SD2

EXECUTIVE SUMMARY

Three structural bugs in CoreBridge.cpp are responsible for nearly all routing quality problems. They are independent of the A* engine (which is correct) and can each be fixed in under 20 lines of code:

BUG 1 Star topology · all pads connect back to pad[0] instead of MST.
Produces the starburst fan pattern visible in every screenshot.

BUG 2 All pads forced to layer 0 · bottom-side SMD pads get an instant via, flooding the board with unnecessary layer-change overhead.

SEVERITY EXPANSION

04:09:43

Close-up: traces collide at random angles, no layer discipline · via cost too low

04:10:28

Mid-zoom: fan pattern emanating from single hub point · star topology confirmed

04:10:46

Full board: every ratsnest line converges to starburst centre · MST missing

pcb_layout.png

Separate board (RP2040 synth): same starburst radiating from one pad

P0 · Critical (breaks routing)

P1 · High (quality / throughput)

P2 · Medium (polish)

EVIDENCE FROM SCREENSHOTS

- 04:09:43

Close-up: traces collide at random angles, no layer discipline · via cost too low
- 04:10:28

Mid-zoom: fan pattern emanating from single hub point · star topology confirmed
- 04:10:46

Full board: every ratsnest line converges to starburst centre · MST missing
- pcb_layout.png

Separate board (RP2040 synth): same starburst radiating from one pad

ROOT CAUSE DETAILS

Star Topology · Missing Minimum Spanning Tree

CoreBridge.cpp:151-153

Routing strategy: for every net, route pad[0] · pad[1], pad[0] · pad[2], ..., pad[0] · pad[N]. This means all N-1 connections share one endpoint. Optimal strategy is MST: connect the closest pair, then the next closest unreached pad, producing a chain/tree not a star. Impact: ~40% excess wire length, massive congestion at hub pad, the starburst pattern.

B1

```
// CURRENT (broken)
auto& start = pts[0];
for (int i = 1; i < pts.size(); ++i) { route(start, pts[i]); }
```

FIX:

Build MST with Prim's algorithm on pad positions before routing.
Route each MST edge instead of hub-spoke edges.
~40 lines in CoreBridge.cpp · no changes to router.cpp needed.

All Pads Hardcoded to Layer 0

CoreBridge.cpp:133

Every pad (SMD top, SMD bottom, through-hole, BGA) is assigned layer 0 regardless of which side the component is placed on. The router then inserts a via immediately on departure for every bottom-side pad, creating a via at every SMD pad on L7.

B2

```
// CURRENT (broken)
netPads[pad.netId].append({px, py}, 0}); // layer always 0
```

FIX:

```
int padLayer = (fp.side == 1) ? (model->copperLayers - 1) : 0;
netPads[pad.netId].append({px, py}, padLayer);
```

Expansion Budget 100x Too Low + Via Cost 6x Too Cheap

CoreBridge.cpp:170-171

maxExpansions=40,000 vs router.h default of 4,000,000. On a dense 8-layer board, A* exhausts this immediately and skips the net, leaving ratsnest lines unrouted. viaCostMm=0.5 vs default 3.0: a via costs less than 2 diagonal moves so the router changes layers every few steps, creating chaotic multi-layer crossing.

B3

```
// CURRENT (broken)
req.viaCostMm = 0.5; // should be 3.0
req.maxExpansions = 40000; // should be 500000+
```

FIX:

```
req.viaCostMm = 3.0;
req.maxExpansions = 500000; // or 2M for power nets
```

Grid Too Coarse · 250µm vs Industry 25µm

CoreBridge.cpp:165

At 0.25mm grid, QFN/TQFP pads at 0.5mm pitch have only 1 grid point between them. The router cannot thread between adjacent pads so it routes over them or fails. KiCad default is 25µm. Even 50µm would give 5 grid points between QFN pads.

B4

```
// CURRENT
req.gridStep = (int64_t)(0.25 * NM); // 250 µm
```

FIX:

```
req.gridStep = (int64_t)(0.05 * NM); // 50µm · 5x finer
Optional: two-pass (100µm rough + 25µm refine) for speed.
```

No Layer Direction Preference (Burstein Model)

router.cpp:135

All 8 directions allowed on every layer. Professional routers assign preferred directions: even layers = horizontal, odd layers = vertical. Diagonal moves dominate because octile heuristic favours them, producing 45° spaghetti cross-layer routing.

B5

```
// CURRENT · all dirs on all layers
const Vec2 dirs[8] = {{g,0},{-g,0},{0,g},{0,-g},{g,g},{g,-g},{-g,g},{-g,-g}};
```

FIX:

Pass preferred axis into isBlocked. Score diagonal moves 2x higher on non-preferred layers. Diagonals become last resort, not first choice.

FIX PRIORITY PLAN

Priority	Fix	Impact	File
P0	MST topology (Prim's)	~40% shorter wire, kills starburst	CoreBridge.cpp:138-153
P0	Fix pad layer assignment	Eliminates spurious via-per-pad	CoreBridge.cpp:133
P0	Expansion=500K, viaCost=3.0	Routes actually complete	CoreBridge.cpp:170-171
P1	Grid 250µm · 50µm	Threads fine-pitch QFN/BGA pads	CoreBridge.cpp:165
P1	Layer direction preference	Structured H/V routing per layer	router.cpp:135
P2	Net ordering by criticality	Power nets take best paths	CoreBridge.cpp:138
P2	Rip-up and reroute (2-pass)	Fewer unrouted nets	CoreBridge.cpp new fn
P3	Differential pair mode	SI compliance on USB/DDR4	router.h/cpp new mode

IMMEDIATE 4-LINE QUICK WIN

These four lines in CoreBridge.cpp alone will produce visibly better routing before touching MST or grid changes. Apply first to prove the direction, then tackle B1.

```
// CoreBridge.cpp · four immediate fixes

// Fix B2: correct pad layer
int padLayer = (fp.side == 1) ? (model->copperLayers - 1) : 0;
netPads[pad.netId].append({{px, py}, padLayer}); // was hardcoded 0

// Fix B3a: restore via penalty
req.viaCostMm = 3.0; // was 0.5 · 6x too cheap

// Fix B3b: enough budget to actually finish routes
req.maxExpansions = 500000; // was 40000 · 100x too low

// Fix B4: finer grid
req.gridStep = (int64_t)(0.05 * NM); // was 0.25mm · 5x too coarse
```

MST ALGORITHM SKETCH (B1 fix)

```
// Replace the star loop in CoreBridge::autoRoute() with Prim's MST:

// Build MST edge list from pad positions
auto mstEdges = [] (const QVector<QPair<QPointF,int>>& pts) {
    // Prim's: start from pts[0], greedily add closest unvisited pad
    std::vector<bool> inMST(pts.size(), false);
    std::vector<std::pair<int,int>> edges;
    inMST[0] = true;
    for (int step = 1; step < (int)pts.size(); ++step) {
        double best = 1e18; int bi = -1, bj = -1;
        for (int i = 0; i < pts.size(); ++i) {
            if (!inMST[i]) continue;
            for (int j = 0; j < pts.size(); ++j) {
                if (inMST[j]) continue;
                double dx = pts[i].first.x()-pts[j].first.x();
                double dy = pts[i].first.y()-pts[j].first.y();
                double d = dx*dx + dy*dy;
                if (d < best) { best=d; bi=i; bj=j; }
            }
        }
        inMST[bj] = true;
        edges.push_back({bi, bj});
    }
    return edges;
};

// Route each MST edge instead of hub-spoke
for (auto [i, j] : mstEdges(pts)) {
    route(pts[i], pts[j], req); // replaces route(start, pts[i])
}
```

COMPARISON: WHY ALTIUM / KICAD / FLUX DO BETTER

Category	Altium / KiCad / Flux	Design Studio (current)
Routing topology	Cost-efficient tree	Star (hub-spoke)
Grid resolution	25-50 µm	250 µm (10x coarser)
Via cost	3.0-5.0 mm equiv.	0.5 mm (6x too cheap)